



hey hey warum muss ich bei einem `flush()`/`commit()` erst reservieren und dann speichern ?

Du musst nicht „erst reservieren und dann speichern“, sondern es passiert genau das im Hintergrund – und `flush()` und `commit()` sind zwei unterschiedliche Schritte dieser Speicherung. ^[1] ^[2]

Was passiert bei `flush()`?

- Dein ORM/Framework (z.B. Hibernate, SQLAlchemy, etc.) hält Änderungen zuerst nur im **Speicher** (Persistence Context / Session), wenn du `save()` oder Objekte änderst. ^[2]
- `flush()` sorgt dafür, dass diese Änderungen als SQL-Befehle an die Datenbank geschickt und ausgeführt werden, damit der Zustand im Speicher und in der DB synchron ist. ^[3] ^[4]
- Wichtig: Nach `flush()` ist die Transaktion noch **nicht** abgeschlossen; die DB kann diese Änderungen noch zurückrollen. ^[4]

Ein typisches Beispiel: Du speicherst ein Objekt und willst direkt danach eine Query ausführen, die die neue Zeile schon berücksichtigen muss (oder du brauchst die von der DB generierte ID in einer Stored Procedure). Dann nutzt man `flush()`, damit die Änderungen sofort „sichtbar“ werden – aber eben nur innerhalb der laufenden Transaktion. ^[2]

Was macht `commit()`?

- `commit()` beendet die Transaktion und sagt der Datenbank: „Alle bisher geflushten Änderungen bitte endgültig übernehmen.“ ^[4]
- Die DB schreibt die Änderungen dauerhaft und danach sind sie für andere Transaktionen sichtbar (außer du arbeitest mit speziellen Isolation Levels wie `READ_UNCOMMITTED`). ^[2] ^[4]
- Ein `commit()` ruft intern meist automatisch ein `flush()` auf, damit vor dem Abschluss der Transaktion alles wirklich rausgeschickt ist. ^[3] ^[4]

Deshalb der Eindruck „erst reservieren, dann speichern“:

- Das „Reservieren“/„Zwischenspeichern“ ist der Zustand im ORM-Cache plus das `flush()` in die laufende Transaktion. ^[3] ^[4]
- Das „Speichern“ im Sinne von **dauerhaft in der Datenbank** ist erst das `commit()`. ^[4]

Warum trennt man das?

- Performance: Viele Änderungen werden gesammelt und in einem Rutsch geflusht/committed, statt für jede Kleinigkeit eine einzelne Transaktion zu machen. ^[5] ^[2]
- Kontrolle: Du kannst festlegen, **wann** deine Änderungen in Queries schon berücksichtigt werden müssen (explizites `flush()`), aber trotzdem noch alles zurückrollen (`rollback`) bis zum `commit()`. ^[2] ^[4]

Kurz gesagt:

- `flush()` → Synchronisiert deinen In-Memory-Zustand mit der DB innerhalb der aktuellen Transaktion. ^[4] ^[2]
- `commit()` → Macht diese geflushten Änderungen endgültig und für andere sichtbar. ^[4]

Wenn du magst, kannst du mir sagen, mit welcher Technologie du arbeitest (Hibernate/Spring Data JPA, SQLAlchemy, etwas anderes), dann erkläre ich dir den Ablauf nochmal genau an deinem konkreten Stack mit einem kleinen Beispielcode.

✱

1. <https://medium.com/@piyush-verma/save-saveandflush-during-transaction-74b041f4355d>
2. <https://www.baeldung.com/spring-data-jpa-save-saveandflush>
3. <https://blog.csdn.net/vernonzheng/article/details/8373999>
4. <https://blog.csdn.net/zmx729618/article/details/51455051>
5. <https://medium.com/javarevisited/understanding-the-importance-of-flushing-in-spring-data-jpa-simplifying-database-synchronisation-266c6c9863b4>
6. <https://thorben-janssen.com/spring-data-jpa-save-saveandflush-and-saveall/>
7. <https://stackoverflow.com/questions/14581865/hibernate-flush-and-commit>
8. <https://medium.com/@firoudreda01/when-does-spring-data-jpa-actually-persist-your-changes-99704cecb9c4>
9. <https://stackoverflow.com/questions/4201455/sqlalchemy-whats-the-difference-between-flush-and-commit>
10. <https://xebia.com/de/blog/jpa-implementation-patterns-saving-detached-entities/>